# RELATÓRIO

## 1. Decisões de projeto

Layout do registro no bloco. Adotei tamanho fixo, com 1512 bytes por registro, resultando em 2 registros por bloco de 4096 e 5 por bloco de 8192. Os campos são: ID (int64, 8 bytes), Título (alfa, 300 bytes), Ano (int, 4 bytes), Citações (int, 4 bytes), Autores (alfa, 150 bytes), Atualização (data/hora, int64, 8 bytes), Snippet (alfa, 1024 bytes), mais padding de alinhamento. A escolha por tamanho fixo se justifica porque permite calcular o offset de qualquer registro dentro do bloco como *slot* × *sizeof(DiskRecord)*, sem precisar de um diretório de offsets variável. Isso é essencial para o par *(bloco, slot)* que as folhas da B+Tree armazenam como valor. O custo dessa decisão é o truncamento: na carga completa, houve 1 título truncado, 552 autores truncados e 1 snippet truncado. A alternativa de tamanho variável com indicador de comprimento tornaria o cálculo do slot O(n) no pior caso e inviabilizaria o acesso direto exigido pela Parte 2.

**Função de hashing, número de buckets e tratamento de colisão**

A função é `bucket = 1 + (id % N)`, com N = 127.679, um primo escolhido no `upload` a partir do número de registros carregados, com meta de ocupação 0,8. O "1 +" reserva o bloco 0 para metadados. As colisões são tratadas por encadeamento em blocos de overflow no fim do arquivo. Na carga completa:

- 29.860 blocos de overflow;
- 26.721 buckets com overflow (≈ 21% do total);
- maior cadeia observada: 4 blocos;
- taxa de ocupação: 64,84%.

A concentração de overflow é maior do que a meta de 0,8 sugeriria, porque o dimensionamento foi feito como `n_registros / 0,8` e não como `n_registros / (0,8 × 2 slots)`. O efeito é visível na seção 7.6.

**Estrutura do nó da B+Tree.**

Construí duas árvores independentes, ambas implementadas em `bptree.hpp` (header-only). Cada nó folha guarda pares `(chave, valor)`, e o valor é um par `(bloco: uint32, slot: uint8)` que aponta para o registro no arquivo de dados.

- Índice primário (ID): chave `int32` de 4 bytes; 340 entradas por folha; ordem de 511 filhos (510 chaves internas); 601 folhas; altura 3.
- Índice secundário (Título, ID): chave composta de 308 bytes (Título 300 + ID 4 + alinhamento); 12 entradas por folha; ordem de 14 filhos (13 chaves internas); 17.024 folhas; altura 5.

**Chaves duplicadas no índice secundário**

A chave composta `(Título, ID)` elimina o problema de duplicatas: o ID desempata títulos repetidos, tornando toda chave única. O `seek2` faz `lower_bound((título, 0))` e varre as folhas encadeadas enquanto o título se mantiver igual ao buscado. Não é necessário tratamento especial de duplicatas dentro do nó.

**Construção dos índices**

Usei bulk load: o `upload` varre o arquivo de dados uma vez, ordena as entradas em memória, monta as folhas já preenchidas e sobe os níveis internos, sem inserção com split. Isso é mais simples e mais rápido do que inserção sequencial, e o custo em blocos escritos está reportado na seção 7.5.

**Contagem de leituras**

O bloco 0 (metadados) do arquivo de dados e dos índices não entra na métrica, seguindo a mesma convenção do arquivo de dados. Como consequência, uma busca exata em `seek1` ou `seek2` lê exatamente `altura` blocos de índice, mais os blocos de dados acessados. Se o `BLOCK_SIZE` compilado no binário não coincidir com o do índice gravado em disco, o programa aborta com mensagem clara, em vez de retornar resultados silenciosamente errados.

**Encoding**

O CSV original contém caracteres UTF-8 fora do ASCII (por exemplo, `Gouvêa`, `Björn`, além de aspas curvas `" "`). Como o layout do registro é byte-oriented, os bytes UTF-8 são armazenados como estão, sem transcodificação. Isso é uma decisão de projeto válida, mas tem uma implicação prática: o cliente que exibe os registros precisa saber que está lendo UTF-8. Não afeta a contagem de blocos, mas explica por que alguns terminais (como o MSYS2 no Windows) travam ao exibir certos trechos.

## 2. Previsão antes da medição

**Ordem e altura esperadas**

Com 204.285 registros carregados:

- Índice primário: $\lceil 204.285 / 340 \rceil = 601$ folhas; altura $\lceil \log_{511}(601) \rceil + 1 = 3$ níveis;
- Índice secundário: $\lceil 204.285 / 12 \rceil = 17.024$ folhas; altura $\lceil \log_{14}(17.024) \rceil + 1 = 5$ níveis.

**Blocos esperados por busca exata**

1. `findrec` (hash): bucket + cadeia de overflow. Como há ~1,6 registros por bucket e o maior overflow tem 4 blocos, espera-se 1 a 2 blocos na maioria dos casos e até 5 no pior caso.
2. `seek1` (primário): exatamente 3 blocos de índice + 1 bloco de dados = 4 no caso de sucesso, 3 no caso de falha.
3. `findtitle`: varredura completa = 157.539 blocos, independentemente do título existir ou não.
4. `seek2` (secundário): 5 blocos de índice + 1 bloco de dados por registro devolvido. Para título inexistente, 5 + 0.
5. `findrange`: varredura completa = 157.539 blocos.
6. `seekrange` (primário): o número de folhas encadeadas percorridas mais 1 bloco de dados por registro devolvido.

**Custos esperados de construção**

- Índice primário: cerca de 2× o número de blocos úteis, ou seja, em torno de 1.200 blocos escritos;
- Índice secundário: cerca de 36.000 blocos escritos.

**Previsão para BLOCK_SIZE = 8192**

- Registros por bloco no arquivo de dados: 5 (contra 2);
- N de buckets: praticamente o mesmo (o N depende do número de registros, não do bloco);
- Entradas por folha do primário: 680; altura cai para 2;
- Entradas por folha do secundário: 24; altura cai para 4;
- Consequentemente, `seek1` deve cair de 4 para 3 blocos no total, e `seek2` de 6 para 5.

## 3. Medição (4096)

Resultados das consultas oficiais com BLOCK_SIZE = 4096

**Busca exata por ID — IDs existentes**

1. ID 5: `findrec` leu 1 bloco; `seek1` leu 3 blocos de índice + 1 de dados = 4. Registro encontrado.
2. ID 316639: `findrec` leu 2 blocos; `seek1` leu 3 + 1 = 4.
3. ID 663752: `findrec` leu 1 bloco; `seek1` leu 3 + 1 = 4.
4. ID 1014283: `findrec` leu 2 blocos; `seek1` leu 3 + 1 = 4.
5. ID 1483294: `findrec` leu 1 bloco; `seek1` leu 3 + 1 = 4.

**Busca exata por ID — IDs inexistentes.**

6. ID 1: `findrec` leu 1 bloco (não encontrado); `seek1` leu 3 + 0 = 3.
7. ID 2000000: `findrec` leu 2 blocos (não encontrado); `seek1` leu 3 + 0 = 3.

**Busca por título**

1. *"Design and implementation of an immersive virtual reality system based on a smartphone platform."* — 1 registro. `findtitle` leu 157.539 blocos; `seek2` leu 5 blocos de índice + 1 de dados = 6.
2. *"A rotating-magnet based wireless electrical power transfer for biomedical implants"* — 1 registro. `findtitle` leu 157.539; `seek2` leu 6 + 1 = 7.
3. *"Review of particle physics particle data group"* — 45 registros. `findtitle` leu 157.539; `seek2` leu 9 + 45 = 54.
4. *"Two-dimensional gas of massless Dirac fermions in graphene"* — 26 registros. `findtitle` leu 157.539; `seek2` leu 7 + 26 = 33.
5. *"Um titulo que nao existe na base"* — 0 registros. `findtitle` leu 157.539; `seek2` leu 5 + 0 = 5.

**Busca por intervalo de IDs**

1. [1, 50] — 10 registros. `findrange` leu 157.539 blocos; `seekrange` leu 3 blocos de índice + 10 de dados = 13.
2. [1000, 1100] — 20 registros. `findrange` leu 157.539; `seekrange` leu 3 + 20 = 23.
3. [500000, 500500] — 99 registros. `findrange` leu 157.539; `seekrange` leu 3 + 99 = 102.
4. [200000, 260000] — 3.052 registros. `findrange` leu 157.539; `seekrange` leu 12 + 3.052 = 3.064.
5. [1600000, 1700000] — 0 registros. `findrange` leu 157.539; `seekrange` leu 3 + 0 = 3.

**Totais dos arquivos**

- Arquivo de dados: 157.540 blocos (incl. 1 de metadados);
- Índice primário: 605 blocos (incl. 1 de metadados);
- Índice secundário: 18.336 blocos (incl. 1 de metadados).

**Custo de construção dos índices**

- Blocos escritos no build do índice primário: 1.210;
- Blocos escritos no build do índice secundário: 36.672;
- Total na carga completa (incluindo o arquivo de dados): 391.686 blocos escritos, em 172.914 ms.

### 3.1. Medição — BLOCK_SIZE = 8192

Resultados das mesmas consultas oficiais sobre o arquivo de dados e índices gerados pela carga com BLOCK_SIZE = 8192.

**Busca exata por ID — IDs existentes.**

1. ID 5: `findrec` leu 1 bloco; `seek1` leu 2 blocos de índice + 1 de dados = 3.
2. ID 316639: `findrec` leu 1 bloco; `seek1` leu 2 + 1 = 3.
3. ID 663752: `findrec` leu 1 bloco; `seek1` leu 2 + 1 = 3.
4. ID 1014283: `findrec` leu 1 bloco; `seek1` leu 2 + 1 = 3.
5. ID 1483294: `findrec` leu 1 bloco; `seek1` leu 2 + 1 = 3.

**Busca exata por ID — IDs inexistentes.**

1. ID 1: `findrec` leu 1 bloco; `seek1` leu 2 + 0 = 2.
2. ID 2000000: `findrec` leu 2 blocos; `seek1` leu 2 + 0 = 2.

**Busca por título.**

1. "Design and implementation..." — 1 registro. `findtitle` leu 61.787; `seek2` leu 4 + 1 = 5.
2. "A rotating-magnet..." — 1 registro. `findtitle` leu 61.787; `seek2` leu 4 + 1 = 5.
3. "Review of particle physics particle data group" — 45 registros. `findtitle` leu 61.787; `seek2` leu 6 + 45 = 51.
4. "Two-dimensional gas of massless Dirac fermions in graphene" — 26 registros. `findtitle` leu 61.787; `seek2` leu 5 + 26 = 31.
5. "Um titulo que nao existe na base" — 0 registros. `findtitle` leu 61.787; `seek2` leu 4 + 0 = 4.

**Busca por intervalo de IDs.**

1. [1, 50] — 10 registros. `findrange` leu 61.787; `seekrange` leu 2 + 10 = 12.
2. [1000, 1100] — 20 registros. `findrange` leu 61.787; `seekrange` leu 2 + 20 = 22.
3. [500000, 500500] — 99 registros. `findrange` leu 61.787; `seekrange` leu 2 + 99 = 101.

4. [200000, 260000] — 3.052 registros. `findrange` leu 61.787; `seekrange` leu 7 + 3.052 = 3.059.
5. [1600000, 1700000] — 0 registros. `findrange` leu 61.787; `seekrange` leu 2 + 0 = 2.

**Totais dos arquivos em 8192.**

1. Arquivo de dados: 61.788 blocos (incl. 1 de metadados).
2. Índice primário: 302 blocos (incl. 1 de metadados).
3. Índice secundário: 8.489 blocos (incl. 1 de metadados).

**Custo de construção em 8192.**

1. Blocos escritos no build do índice primário: 604.
2. Blocos escritos no build do índice secundário: 16.978.
3. Blocos escritos na carga completa (dados + índices): 276.752.
4. Tempo de carga: 91.276 ms.

### 4. Comparação e explicação das divergências

Comparação com o previsto na seção 7.2. As previsões de altura se confirmaram integralmente: 3 e 5 níveis em 4096, 2 e 4 níveis em 8192. As contagens de folhas também: 601 e 300 para o primário, 17.024 e 8.172 para o secundário. As divergências concentram-se em dois pontos: o custo observado de `findrec` e a variação de `seek2` conforme a posição do título.

Sobre a previsão do número de folhas. As previsões 601/300 e 17.024/8.172 se confirmaram por coincidirem com ⌈n / entradas_por_folha⌉. O bulk load fez com que todas as folhas fossem preenchidas exatamente no limite de capacidade, com a última folha possivelmente parcial.

Índice primário na busca exata. Em 4096, `seek1` leu exatamente 3 blocos de índice em todos os 7 casos, com sucesso ou não. Em 8192, exatamente 2. O `findrec`, em compensação, variou entre 1 e 2 blocos nos dois cenários, com média de ~1,4. Para IDs existentes, o hash ganha com folga (1-2 blocos contra 4 em 4096 e 3 em 8192). Para IDs inexistentes, o índice ganha (3 ou 2 blocos, com corte na descida), mas a diferença é pequena. Esse é o fenômeno que o enunciado antecipa: para busca exata por ID, a organização hash pode ser mais barata que a B+Tree.

Índice secundário na busca por título. Em 4096, `seek2` variou de 5 a 6 blocos de índice; em 8192, de 4 a 6. A explicação é que o bulk load bottom-up não balanceia perfeitamente a profundidade de todos os ramos. Algumas folhas ficam em ramos de altura 5, outras em ramos de altura 6 (4096), e algumas em ramos de altura 4,

outras em 5 (8192). Isso não é um bug: é uma característica esperada do algoritmo. Se a construção fosse por inserção sequencial com splits, o balanceamento seria mais uniforme, mas o custo de construção seria muito maior.

Variação do custo de `seek2` conforme o número de correspondências.

1. Título único, 4096: 5 ou 6 blocos de índice. Título único, 8192: 4 blocos.
2. Título com 45 ocorrências, 4096: 9 blocos de índice. Em 8192: 6 blocos. A diferença reflete o número de folhas extras varridas (mais em 4096, porque cada folha cabia 12 entradas; menos em 8192, porque cada folha cabe 25).
3. Título com 26 ocorrências, 4096: 7 blocos. Em 8192: 5 blocos.
4. Título inexistente, 4096: 5 blocos de índice; em 8192: 4 blocos. O custo aqui é apenas a descida até a folha onde o título "deveria" estar.

`findtitle` e `findrange` são indiferentes ao resultado. Em todos os títulos e intervalos testados, `findtitle` e `findrange` leram exatamente o arquivo inteiro: 157.539 blocos em 4096 e 61.787 blocos em 8192. Não há variação com o número de registros encontrados.

`seekrange` ganha, mas o ganho cai com o tamanho do intervalo. As razões de ganho entre `findrange` e `seekrange` (em blocos totais lidos) foram:

1. [1, 50]: 157.539 / 13 ≈ 12.118× em 4096; 61.787 / 12 ≈ 5.149× em 8192.
2. [1000, 1100]: 157.539 / 23 ≈ 6.850× em 4096; 61.787 / 22 ≈ 2.808× em 8192.
3. [500000, 500500]: 157.539 / 102 ≈ 1.544× em 4096; 61.787 / 101 ≈ 612× em 8192.
4. [200000, 260000]: 157.539 / 3.064 ≈ 51× em 4096; 61.787 / 3.059 ≈ 20× em 8192.
5. [1600000, 1700000]: 157.539 / 3 ≈ 52.513× em 4096; 61.787 / 2 ≈ 30.894× em 8192.

O ganho absoluto do `seekrange` sobre `findrange` cai com o tamanho do intervalo, porque `seekrange` paga 1 bloco de dados por registro devolvido (sem cache), e esse custo converge para o custo da varredura completa conforme o número de registros aumenta. No limite de um intervalo cobrindo todos os IDs, `seekrange` e `findrange` teriam custo comparável.

Divergências identificadas.

1. A altura do índice secundário não é uniforme em todos os ramos. Explicação: bulk load bottom-up não garante balanceamento de profundidade perfeito.
2. A taxa de ocupação real (64,84% em 4096, 66,13% em 8192) ficou abaixo da meta de 80%, porque o dimensionamento do N usou `n_registros / 0,8` em vez de `n_registros / (0,8 × slots_por_bloco)`. Consequência: 21% dos

buckets têm overflow em 4096, com cadeias de até 4 blocos, o que explica por que `findrec` por vezes leu 2 blocos.

3. O número de buckets caiu de 127.679 para 51.109 com o aumento do bloco. Isso é surpreendente à primeira vista, mas se explica porque o cálculo de N foi feito em função do número de registros e do número de registros por bloco (para garantir que o arquivo de dados acomode N buckets sem overflow imediato). Dobrar o tamanho do bloco dobra a capacidade de registros por bucket e, portanto, reduz o N necessário.